ASIF SAAS STUDIO

Image to PDF Generator

Complete documentation: a detailed how-to-use guide, settings reference, the built-in PDF engine explained, validation rules, deployment and testing for the browser-only image-to-PDF application.

Creator: Dr. Asif Ahmad
Version: 1.0 · **Category:** Design
Date: 20 October 2026
Deliverable: a single self-contained `index.html` (22.9 KB) — no server, no build step, no accounts, no uploads.

1 · Introduction

Image to PDF Generator turns a set of photos into one clean, print-ready PDF — entirely inside the visitor's browser. The user adds images, arranges their order, chooses a page size, and downloads a real PDF document. Nothing is uploaded anywhere.

The application was built for the Asif SaaS Studio tools catalogue, where every tool runs inside a sandboxed iframe on a Wix Headless site. That environment forbids API keys, browser storage, service workers and permission prompts — so the app is a single self-contained HTML file whose PDF engine is written by hand in JavaScript: no PDF library, no backend, no network requests. It produces standard PDF 1.4 documents that open in every reader, on every platform.

Typical uses

- Students combining photographed notes, assignments or whiteboard pictures into one submittable PDF.
- Teachers assembling scanned worksheets and handouts for a class.
- Offices turning phone photos of receipts, forms and signed pages into a single document for email.
- Anyone who needs a quick, private image-to-PDF conversion without handing photos to an unknown server.

> **Privacy first.** Because conversion happens 100% on the user's own device, Image to PDF Generator is safe for personal documents — IDs, letters, medical or financial photos — that should never be uploaded to a conversion website.

2 · Features at a glance

Feature	What it does
Up to 50 images per PDF	Each image becomes one page, in exactly the order shown on screen.
Wide format support	PNG, JPEG, WebP, BMP, GIF (first frame) and AVIF inputs, up to 25 MB each.
Drag & drop + browse	Drop zone with keyboard access, plus a multi-select file picker. Files can be added in several batches.
Reorder & remove	Every thumbnail row carries ↑ / ↓ buttons and a remove (✕) button; numbering updates instantly.
Page sizes	A4, Letter, or Fit-to-image (the page adopts each photo's own shape).
Orientation	Auto (follows each image), Portrait or Landscape.
Margins & quality	Margins 0–25 mm; JPEG quality 60–100 to balance sharpness against file size.
Custom file name	Name the PDF before generating; unsafe characters are cleaned automatically.
Hand-written PDF engine	Generates genuine PDF 1.4 files (JPEG image streams, exact cross-reference table, document title metadata) with zero libraries.
Full input validation	Wrong types, empty and corrupt files, oversize files and the 50-image cap are each explained in plain language, naming the file at fault.

3 · How to use it — step by step

Step 1 — Open the tool

Open `index.html` in any modern browser (Chrome, Edge, Firefox, Safari), or open *Image to PDF Generator* from the Asif SaaS Studio tools catalogue. There is nothing to install and no account to create. The screen has two panels: **“1 · Add your images”** on the left and **“2 · PDF settings”** on the right (on a phone they stack, images first).

Step 2 — Add your images

Two ways, both supporting several files at once:

- **Drag & drop:** drag image files from your computer or phone gallery onto the dashed drop zone and release. The zone lights up while you hover.
- **Browse:** tap or click the drop zone (or press Enter/Space while it is focused) and select files in the picker. Hold Ctrl (Windows) or Cmd (Mac) to pick several at once.

You can add images in *several batches* — each new batch is appended to the list. Every accepted image appears as a numbered row showing a thumbnail, the file name, its pixel dimensions and its size. If a file is refused, a red message names that file and says exactly why (wrong type, empty, too large, or unreadable) — the other files are still added.

> **Order matters.** Page 1 of the PDF is the first row in the list, page 2 the second, and so on. Check the numbering before you generate.

Step 3 — Arrange the order (optional)

- Tap ↑ or ↓ on any row to move that image one position up or down. The row numbers refresh immediately.
- Tap ✕ to remove an image you added by mistake.
- Use **Clear all** (in the settings panel) to empty the list and start again.

Step 4 — Choose the page size

- **A4** — the standard for documents, assignments and printing in most of the world. Choose this unless you have a reason not to.
- **Letter** — the North-American standard (8.5 × 11 in).
- **Fit to image** — every page is shaped exactly like its photo (plus the chosen margin). Best for photo sets and artwork where cropping the page shape to the picture looks cleanest.

Step 5 — Choose the orientation

- **Auto (recommended)** — a wide photo gets a landscape page; a tall photo gets a portrait page. Each page follows its own image.
- **Portrait** — forces upright pages for all images (images are still never stretched; they are fitted inside the page).
- **Landscape** — forces wide pages for all images.

Step 6 — Set the margin and quality

- **Page margin (0–25 mm, default 10 mm):** white space around each image. 10 mm looks right for most documents and keeps images clear of printer edges. Use 0 mm for full-bleed photo pages (with Fit-to-image).
- **Image quality (60–100, default 90):** how strongly the photos are compressed inside the PDF. 90 is visually lossless for normal photos. Drop to 75–80 when you must email a large set and file size matters; raise to 100 for maximum fidelity.

Step 7 — Name the file

Type a name in the **PDF file name** box (for example `Assignment-Unit-3`). The `.pdf` ending is added for you; characters that are unsafe in file names are removed automatically, and an empty box falls back to `my-images.pdf`. The name is also written inside the PDF as its document title.

Step 8 — Generate and download

1. Tap 📄 **Generate PDF**. The button shows “Building PDF…” while the pages are assembled — a few seconds for large sets.
2. The result panel appears: **“3 · Your PDF”**, reporting the exact page count and file size.
3. Tap ⬇ **Download PDF** and save the file. Open it in any PDF reader to check the pages before sharing.

Worked example — photographed class notes

You photographed 6 pages of handwritten notes with your phone (tall JPEG photos). Add all 6 → check the order matches the pages → Page size **A4**, Orientation **Auto** (each note stays portrait), Margin **10 mm**, Quality **90** → name it `Physics-Notes-Week-4` → Generate → Download. The result is a tidy 6-page A4 PDF ready to submit or print.

> **Transparent images.** PNGs with transparency (logos, screenshots with alpha) are placed on a *white* background in the PDF, because PDF pages are white — transparent areas never turn black or checkerboard.

4 · Settings reference

Setting	Options / range	Default	Notes
Page size	A4 · Letter · Fit	A4	Fit ignores A4/Letter geometry and shapes each page to its image (at 96 dpi) plus margins.
Orientation	Auto · Portrait · Landscape	Auto	Applies to A4/Letter only; Fit pages follow the image by definition.
Margin	0–25 mm in 5 mm steps	10 mm	Converted internally to points (mm × 72 / 25.4).
Quality	60–100	90	JPEG encoding quality for the embedded images.
File name	Up to 60 characters	my-images	Sanitised automatically; also stored as the PDF's Title metadata.
Images	1–50 files, ≤ 25 MB each	—	PNG, JPEG, WebP, BMP, GIF (first frame), AVIF.

5 · Input validation

Every file is checked individually, before and during loading; one bad file never blocks the good ones.

Check	Rule and behaviour
Supported type	Allow-list: PNG, JPEG, WebP, BMP, GIF, AVIF. Anything else is refused with a message naming the file.
Empty file	0-byte files are refused: “appears to be empty”.
File size	Over 25 MB is refused before decoding, protecting device memory.
Count	At most 50 images; further files are refused with the cap explained.
Unreadable / corrupt	If the browser cannot decode a file (or it reports no dimensions), it is skipped and named in the message.
No images	The Generate button stays disabled until at least one valid image is in the list.

6 · How it works — the built-in PDF engine

The app contains a small, purpose-written PDF 1.4 writer. It exists because conventional PDF libraries are far larger than this app's 150 KB budget — and because a converter that uploads your documents to a server was never acceptable.

1. **Decode.** Each image is decoded by the browser and its true pixel dimensions recorded.
2. **Geometry.** For every image, the page size is resolved (preset + orientation + margins) and the image is fitted — aspect preserved, centred — into the page's margin box.
3. **Encode.** The image is drawn once onto a white canvas (flattening transparency) and encoded as a JPEG at the chosen quality. JPEG bytes are embedded in the PDF *as-is* using the DCTDecode filter — no double compression, no quality loss beyond the chosen setting.
4. **Assemble.** The writer emits the catalogue, page tree, one page object + content stream + image object per picture, and a document-information object carrying the title. Byte offsets are tracked as it writes, so the cross-reference table is exact — the property strict PDF readers check first.
5. **Deliver.** The finished bytes become a downloadable file entirely in memory; nothing touches a server or disk cache.

The drawing instruction on each page is a single PDF transformation: the unit image space is scaled to the fitted rectangle (`w 0 0 h x y cm`) and painted with one operator. Page geometry uses points throughout: A4 = 595.28 × 841.89 pt, Letter = 612 × 792 pt; Fit pages measure the image at 96 dpi (px × 72/96) plus margins.

7 · Architecture & sandbox compliance

One HTML file (22,873 bytes) in three layers: presentation (inline CSS, responsive two-column grid, studio palette — emerald `#103E32`, lime `#DCF5B5`, paper `#F7F8F2`), interface (drop zone, thumbnail list, settings, result panel), and a **pure core** with no DOM access — validation, page geometry, fitting, and the PDF byte builder (`validateImageFile`, `pageDims`, `fitRect`, `buildPdfBytes`) — which is unit-tested directly (Section 9).

Sandbox rule	How the app complies
No API keys or private data	None exist anywhere in the file.
No localStorage / cookies	Never used; state lives in ordinary variables for the session only.
No service workers	None registered; works from `file://` or any static host.
No browser permissions	No camera, microphone, notification, geolocation or clipboard APIs. Files arrive only through the user's own picker or drop.
No parent-window access	`window.parent` and `postMessage` are never used; behaviour is identical standalone or embedded.
No network calls	Zero external assets, zero fetches. Downloads use in-memory object URLs, revoked when replaced or cleared.

8 · Deployment

8.1 Asif SaaS Studio (Wix Headless + CMS)

1. Wix dashboard → CMS → **SaasProducts** → add a new item.
2. Title: *Image to PDF Generator — Convert Photos to PDF*; Slug: `image-to-pdf`; Category: *Design*; Version: *1.0*; Demo: Yes; Published: Yes.
3. Paste the complete contents of `index.html` into the **appHtml** field.
4. Open `app.html?slug=image-to-pdf` on the live site and run the smoke test in Section 9.3.

8.2 Any other host

Upload `index.html` to any static host or open it from disk. No build step, no configuration, no environment variables.

9 · Testing

9.1 Automated tests — 25/25 passed

The pure core was extracted from the shipped file and executed in Node.js; a real two-page PDF produced by that exact code was then verified independently with poppler (`pdfinfo` / `pdftoppm`).

Area	What was verified	Result
Geometry (7 tests)	A4/Letter exact point sizes; Auto orientation both ways; Fit sizing with margins; fit-rectangle aspect and centring for wide and tall images.	Pass
Text safety (5 tests)	PDF number formatting; file-name sanitising incl. extension stripping and defaults; title escaping.	Pass
Validation (5 tests)	Valid image passes; wrong type, empty file, 26 MB file and the 51st image are all refused.	Pass
PDF structure (8 tests)	Header/EOF markers; page count; image stream lengths match real JPEG bytes; **every cross-reference offset points exactly at its object**; startxref accuracy; trailer size.	Pass
Independent reader check	Poppler reads the generated file: 2 pages, correct A4 landscape size, title metadata returned exactly; both pages render without structural errors.	Pass

The complete inline script also passes a JavaScript syntax check, and the file size was verified at build time (22.9 KB against the 150 KB ceiling).

9.2 Manual checklist

- Open `index.html` in Chrome/Edge — loads cleanly, no console errors.
- Add 3 photos → thumbnails appear with dimensions; reorder with ↑ ↓ and confirm the page order in the PDF.
- Drop a `.pdf` or `.txt` file → clear “not a supported image” message naming the file.
- Add a file over 25 MB → size message; add a corrupt image → “could not be read” message.
- Generate on A4/Auto → images centred, orientation follows each photo.
- Fit-to-image with a tall phone photo → the page matches the photo's shape.
- Margin 0 vs 25 mm → visible difference; quality 60 vs 100 → file size changes as expected.
- Transparent PNG → appears on white in the PDF.
- Narrow window (~380 px) → layout stacks; every control reachable.

9.3 Live smoke test (after CMS publishing)

On the live page, convert two known photos on A4/Auto and confirm: the app loads inside the iframe, the download saves, the PDF has exactly 2 pages, and the second photo is on page 2.

10 · Troubleshooting & FAQ

Question	Answer
“Not a supported image”	The file is not PNG/JPEG/WebP/BMP/GIF/AVIF. Convert it first — a screenshot saved as PNG always works.
A photo is missing from the PDF	It was probably refused at add-time or removed. The red message after adding names every refused file and why.
Pages are in the wrong order	Reorder with ↑ ↓ before generating — the row numbers are the page numbers.
The PDF file is very large	Lower Image quality to 75–80 and regenerate; photo-heavy PDFs shrink dramatically with little visible difference.
Images look cropped	They never are — images are always fitted whole inside the page. What changes is the white margin around them; try Fit-to-image or a smaller margin.
My GIF only shows one frame	By design: animated GIFs contribute their first frame, since PDF pages are static.
Text in photos is slightly soft	Raise Quality to 95–100. The embedded image is exactly as sharp as the setting allows.
Is anything uploaded?	No. The PDF is assembled on your device; the page makes no network requests at all.
The download didn't start in the embedded page	Open the tool in a full browser tab — some embedded contexts restrict downloads.

11 · Limitations

- One image per page; there is no multi-image collage layout in v1.0.
- No OCR: photographed text stays a picture — it is not converted to selectable text (that would require a recognition engine/backend).
- No PDF editing: existing PDFs cannot be merged or split by this tool.
- Output PDFs are uncompressed in structure (image streams use JPEG compression only); extremely large sets can produce large files — the quality slider is the control.
- Animated formats contribute their first frame only.

12 · Version history & credits

Version	Date	Notes
1.0	20 Oct 2026	First release: up to 50 images to one PDF; A4/Letter/Fit pages; Auto orientation; margins and quality controls; reorder/remove; hand-written PDF 1.4 engine; full validation. 25/25 automated tests passed, generated output verified in an independent PDF reader.

Designed and built for **Asif SaaS Studio**.

Creator: Dr. Asif Ahmad

The application contains no third-party code, libraries or assets.

Appendix A · The pure core (verbatim)

These are the exact functions tested in Section 9 — validation, page geometry and the complete PDF byte builder — reproduced from the shipped `index.html`. The complete file, interface included, ships as `index.html` with the product.

```
var IMG_TYPES = ["image/png", "image/jpeg", "image/webp", "image/bmp", "image/gif", "image/avif"];
var MAX_FILE_BYTES = 25 * 1024 * 1024, MAX_IMAGES = 50;

// Validate one incoming file. Returns "" when valid, else a human-readable reason.
function validateImageFile(file, currentCount){
  if (!file) return "No file was provided.";
  if (currentCount >= MAX_IMAGES) return "You can add at most " + MAX_IMAGES + " images.";
  if (IMG_TYPES.indexOf(file.type) === -1)
    return "“" + (file.name || "file") + "” is not a supported image (use PNG, JPEG, WebP, BMP, GIF or
AVIF).";
  if (file.size === 0) return "“" + (file.name || "file") + "” appears to be empty.";
  if (file.size > MAX_FILE_BYTES) return "“" + (file.name || "file") + "” is larger than 25 MB.";
  return "";
}

// Page dimensions in points (1 pt = 1/72 in) for a preset + orientation + image shape.
function pageDims(preset, orientation, imgW, imgH, marginPt){
  var w, h;
  if (preset === "letter"){ w = 612; h = 792; }
  else if (preset === "fit"){ w = imgW * 0.75 + 2 * marginPt; h = imgH * 0.75 + 2 * marginPt; return { w: w,
h: h }; }
  else { w = 595.28; h = 841.89; } // A4
  var landscape;
  if (orientation === "landscape") landscape = true;
  else if (orientation === "portrait") landscape = false;
  else landscape = imgW > imgH; // auto
  if (landscape && w < h){ var t = w; w = h; h = t; }
  if (!landscape && h < w){ var t2 = w; w = h; h = t2; }
  return { w: w, h: h };
}

// Fit an image inside the page's margin box, preserving aspect, centred.
function fitRect(imgW, imgH, pageW, pageH, marginPt){
  var boxW = Math.max(1, pageW - 2 * marginPt), boxH = Math.max(1, pageH - 2 * marginPt);
  var s = Math.min(boxW / imgW, boxH / imgH);
  var w = imgW * s, h = imgH * s;
  return { x: (pageW - w) / 2, y: (pageH - h) / 2, w: w, h: h };
}

// Format a PDF number: at most 2 decimals, never exponential.
function pdfNum(n){
  var v = Math.round(n * 100) / 100;
  return (v % 1 === 0) ? String(v) : String(v);
}

// Keep a PDF string literal safe (title metadata).
function pdfEscape(s){
  return String(s).replace(/\\/g, "\\\\").replace(/\(/g, "\\(").replace(/\)/g, "\\)")
    .replace(/[^\x20-\x7E]/g, " ");
}

// Sanitize a download file name (without extension).
function safeBaseName(name){
  var base = String(name || "").replace(/\.pdf$/i, "").replace(/[^\w\- ]+/g, "").trim();
  return base ? base.slice(0, 60) : "my-images";
}

// Build a complete PDF (as bytes) from prepared pages.
// page = { jpeg:Uint8Array, imgW, imgH, pageW, pageH, x, y, w, h }
function buildPdfBytes(pages, title){
  var enc = new TextEncoder();
  var parts = [], offsets = {}, size = 0;
  function push(bytes){ parts.push(bytes); size += bytes.length; }
  function pushStr(s){ push(enc.encode(s)); }
  function beginObj(id){ offsets[id] = size; pushStr(id + " 0 obj\n"); }

  var n = pages.length;
  var infoId = 3 + 3 * n;
```

```
  var total = infoId + 1;

  pushStr("%PDF-1.4\n");
  push(new Uint8Array([0x25, 0xE2, 0xE3, 0xCF, 0xD3, 0x0A])); // binary marker comment

  beginObj(1);
  pushStr("<< /Type /Catalog /Pages 2 0 R >>\nendobj\n");

  var kids = [];
  for (var i = 0; i < n; i++) kids.push((3 + 3 * i) + " 0 R");
  beginObj(2);
  pushStr("<< /Type /Pages /Kids [" + kids.join(" ") + "] /Count " + n + " >>\nendobj\n");

  for (var p = 0; p < n; p++){
    var pg = pages[p], pageId = 3 + 3 * p, contentId = pageId + 1, imageId = pageId + 2;
    var content = "q\n" + pdfNum(pg.w) + " 0 0 " + pdfNum(pg.h) + " " + pdfNum(pg.x) + " " +
      pdfNum(pg.y) + " cm\n/Im0 Do\nQ\n";
    beginObj(pageId);
    pushStr("<< /Type /Page /Parent 2 0 R /MediaBox [0 0 " + pdfNum(pg.pageW) + " " + pdfNum(pg.pageH) +
      "] /Resources << /XObject << /Im0 " + imageId + " 0 R >> >> /Contents " + contentId + " 0 R
>>\nendobj\n");
    beginObj(contentId);
    pushStr("<< /Length " + content.length + " >>\nstream\n" + content + "endstream\nendobj\n");
    beginObj(imageId);
    pushStr("<< /Type /XObject /Subtype /Image /Width " + pg.imgW + " /Height " + pg.imgH +
      " /ColorSpace /DeviceRGB /BitsPerComponent 8 /Filter /DCTDecode /Length " + pg.jpeg.length + "
>>\nstream\n");
    push(pg.jpeg);
    pushStr("\nendstream\nendobj\n");
  }

  beginObj(infoId);
  pushStr("<< /Title (" + pdfEscape(title || "Images") + ") /Creator (Asif SaaS Studio - Image to PDF
Generator) /Producer (Image to PDF Generator v1.0) >>\nendobj\n");

  var xrefPos = size;
  var xref = "xref\n0 " + total + "\n0000000000 65535 f \n";
  for (var id = 1; id < total; id++){
    var off = String(offsets[id]);
    while (off.length < 10) off = "0" + off;
    xref += off + " 00000 n \n";
  }
  pushStr(xref);
  pushStr("trailer\n<< /Size " + total + " /Root 1 0 R /Info " + infoId + " 0 R >>\nstartxref\n" + xrefPos +
"\n%%EOF");

  var out = new Uint8Array(size), pos = 0;
  for (var k = 0; k < parts.length; k++){ out.set(parts[k], pos); pos += parts[k].length; }
  return out;
}
```